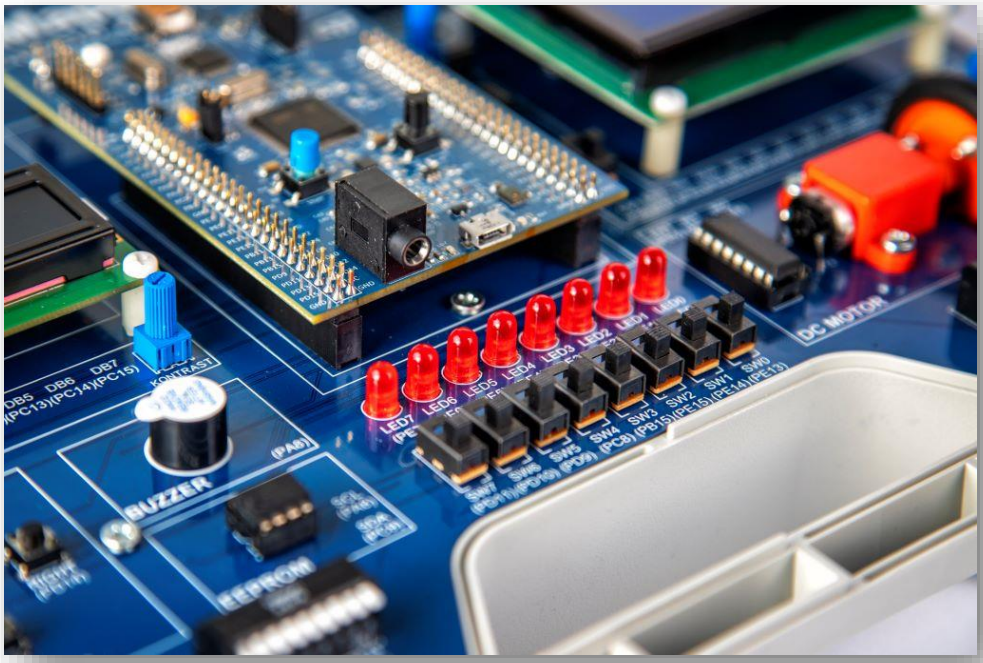
SİVAS CUMHURİYET ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GÖMÜLÜ SİSTEMLER DERSİ

Gömülü Sistemler Deney 1: GPIO ve Dijital Giriş/Çıkış Uygulamaları

STM32CubeIDE ile LED Animasyonu ve Buton Kontrolü (Kara Şimşek Projesi)



Dersin Hocası: Doç. Dr. Ahmet Gürkan YÜKSEK
Dersin Asistanı: Arş. Gör. Ahmet Utku ELİK
Öğrenci Adı Soyadı – Numarası:

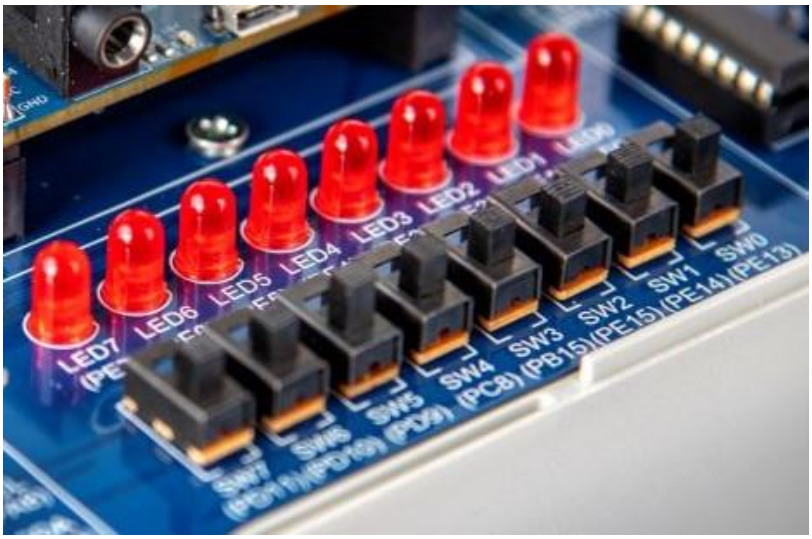
1.Deneyin Amacı:

Bu deneyde, STM32F407 mikrodenetleyicisinin **GPIO (General Purpose Input Output)** birimini kullanarak dijital çıkış (LED kontrolü) ve dijital giriş (Anahtar/Buton okuma) işlemlerinin öğrenilmesi amaçlanmaktadır.

2. Kullanılacak Donanım ve Pin Bağlantıları:

ArmApp18 eğitim seti üzerinde, mikrodenetleyici ile çevre birimleri donanımsal olarak bağlıdır. Bu deneyde giriş ve çıkış işlemleri için **PORT E** kullanılacaktır.

Bileşen	Pin Kodu	Mod (Mode)	Açıklama
LED 0 - 7	PE0 - PE7	GPIO_Output	8 adet kırmızı LED (Çıkış)
Switch 0 (SW0)	PE13	GPIO_Input	Tüm LED'leri Yakma (Set All)
Switch 1 (SW1)	PE14	GPIO_Input	Animasyonu Durdurma (Pause)
Switch 2 (SW2)	PE15	GPIO_Input	Hız Artırma (Speed Up)



Çıkış Modları (Push-Pull mu, Open-Drain mi?)

Bir pini "Çıkış" (Output) olarak ayarladığımızda mikrodenetleyici bize iki seçenek sunar:

1. **Push-Pull (İt-Çek):** Pin, aktif olarak hem **0V (GND)** hem de **3.3V (VCC)** verebilir. LED yakmak gibi güç gerektiren standart işler için **bu mod kullanılır**.

2. **Open-Drain (Açık Akaça):** Pin sadece **0V** verebilir, 3.3V veremez (havada kalır). Lojik 1 için dışarıdan direnç gerekir. Genellikle I2C haberleşmesi veya farklı voltaj seviyelerindeki cihazları kontrol etmek için kullanılır. *Biz bu deneyde LED süreceğimiz için **Push-Pull** modunu kullanacağız.*

TEKNİK DETAY: Giriş Modları (Pull-Up ve Pull-Down) Bir butonu okurken, butona basılmadığı andaki durumu kararlıdır. Ancak basılmadığında pin boşta kalırsa ne olur?

- **Pull-Up (Yukarı Çekme):** Butona basılı değilken pini direnç üzerinden sürekli **3.3V (Lojik 1)** seviyesinde tutar.
- **Pull-Down (Aşağı Çekme):** Butona basılı değilken pini direnç üzerinden sürekli **0V (Lojik 0)** seviyesinde tutar.
- **Not:** ArmApp18 setindeki switchlerde harici dirençler bulunduğu için yazılımda **"No Pull-up/No Pull-down"** seçeneğini kullanacağız.

DİKKAT: Floating (Yüzen) Pin Tehlikesi

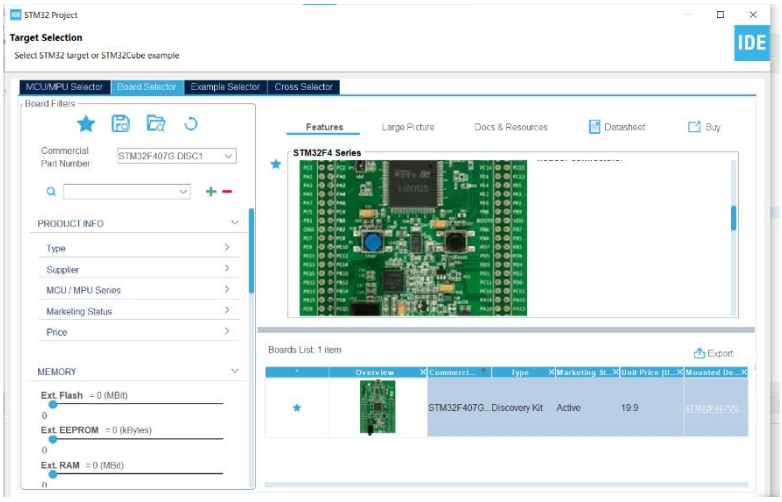
Eğer bir giriş pinine ne Pull-Up ne de Pull-Down direnci bağlamazsanız, o pin **"Floating"** (Yüzen) durumunda kalır. Havadaki elektromanyetik dalgalardan (hatta elinizi yaklaştırmanızdan bile) etkilenerek rastgele **1** veya **0** değerlerini okuyabilir. Bu da projenizin kendi kendine çalışmasına veya kararsız davranmasına neden olur.

3. STM32CubeIDE Konfigürasyonu (CubeMX Ayarları)

Kodlamaya geçmeden önce mikrodenetleyicinin pinlerini ve sistem saatini ayarlamamız gerekmektedir.

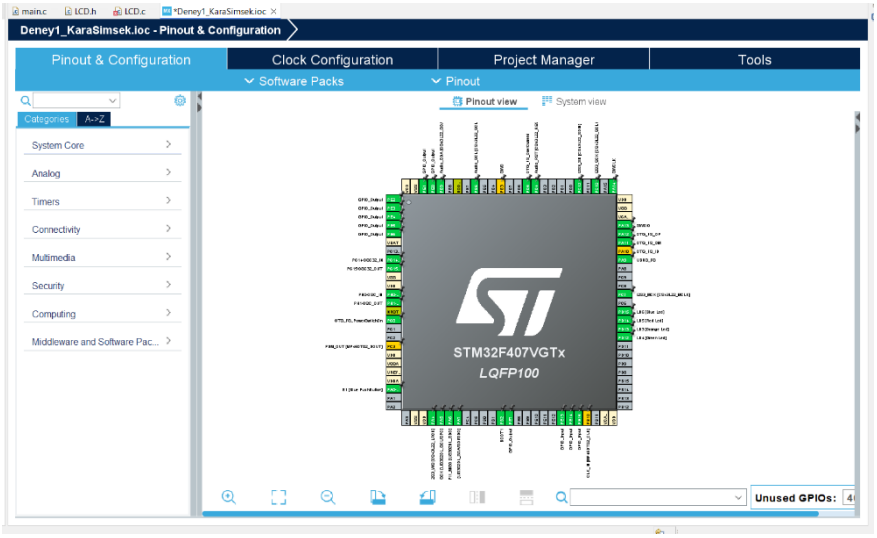
Adım 1: Proje Oluşturma

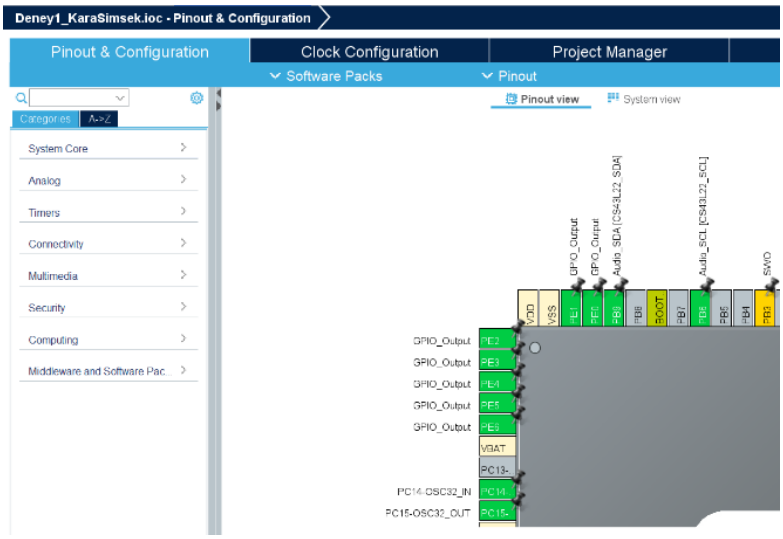
- STM32CubeIDE'yi açın ve File > New > STM32 Project yolunu izleyin.
- **Part Number** arama kutusuna **STM32F407VG** yazın.
- Sağdaki listeden işlemciyi seçip **"Next"** butonuna basın.
- Proje ismine **"Deney1_KaraSimsek"** (veya istediğiniz bir isim) yazıp **"Finish"** deyin.



Adım 2: Pin Kurulumu (Pinout Configuration)
Sağ taraftaki işlemci görseli üzerinden aşağıdaki pinleri bulup sol tıklayarak ilgili modları seçin:

1. **LED Pinleri (Çıkış):**
   - PE0 ... PE7 (Toplam 8 pin) -> **GPIO_Output** seçin.
   - *Pinler yeşil renge dönecektir.*
2. **Switch Pinleri (Giriş):**
   - PE13, PE14, PE15 -> **GPIO_Input** seçin.





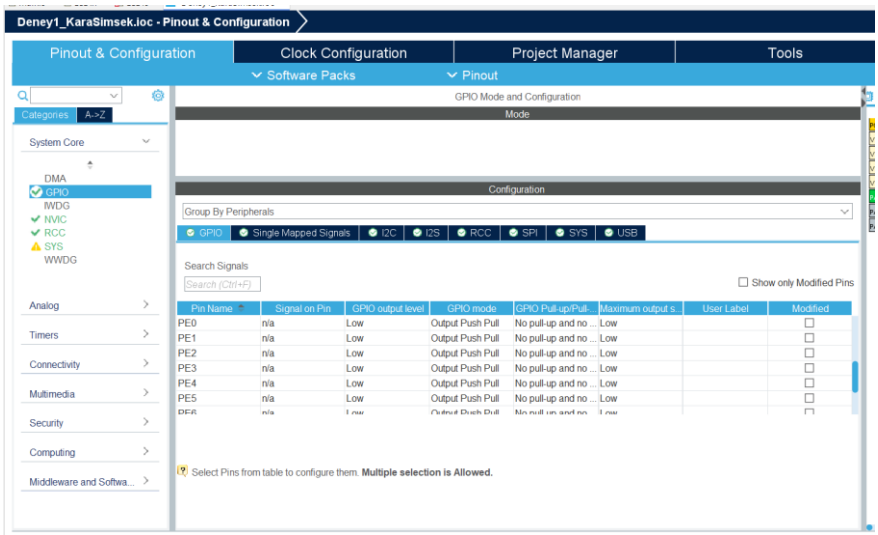
İPUCU: User Label (Kullanıcı Etiketi) Kullanımı Kod yazarken pin numaralarını ezberlemek zordur. Bunun yerine pinlere isim verebiliriz.

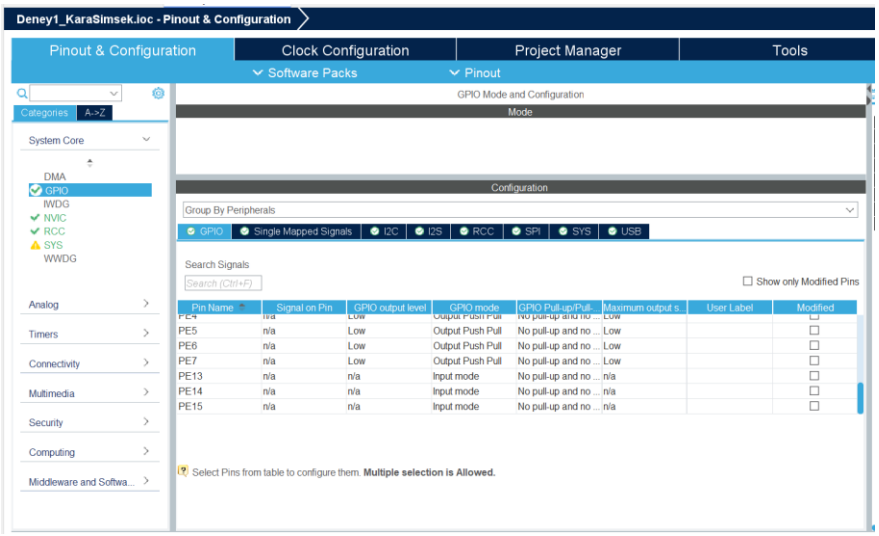
- **System Core > GPIO** sekmesine gelin.
- PE0 pinine tıklayın ve aşağıdan **User Label** kısmına LED0 yazın.
- Bunu tüm LED'ler (LED1...LED7) ve Switch'ler (SW0, SW1, SW2) için yaparsanız kod yazarken işiniz çok kolaylaşır.

Adım 3: GPIO Detaylı Ayarları

Soldaki menüden **System Core > GPIO** sekmesine tıklayın ve ortadaki listeden pinleri seçerek aşağıdaki ayarları doğrulayın :

- **LED Pinleri (PE0 - PE7) İçin:**
  - GPIO Output Level: **Low** (Başlangıçta sönük olsun)
  - GPIO Mode: **Output Push Pull**
  - Pull-up/Pull-down: **No Pull-up and No Pull-down**
  - Maximum Output Speed: **Low** (LED yakmak için yüksek hıza gerek yok)
- **Switch Pinleri (PE13 - PE15) İçin:**
  - GPIO Mode: **Input Mode**
  - Pull-up/Pull-down: **No Pull-up and No Pull-down** (Set üzerinde harici direnç olduğu için)

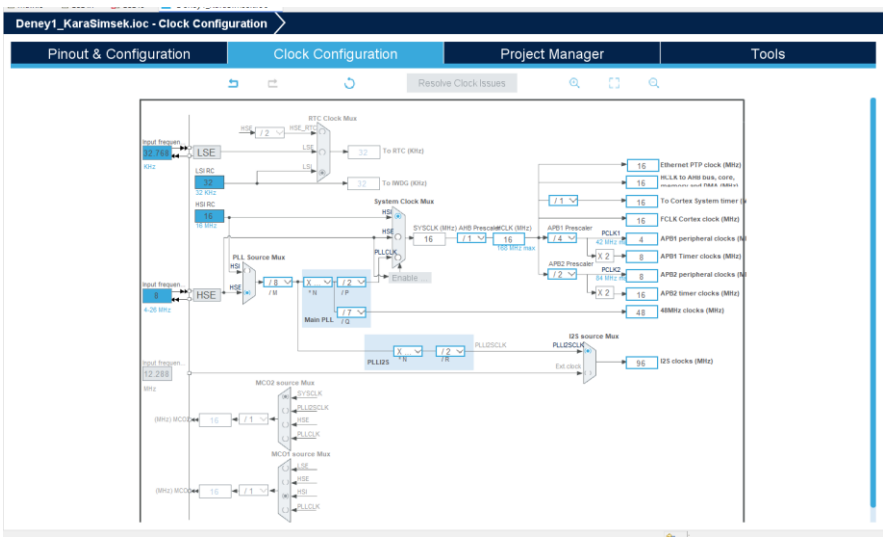


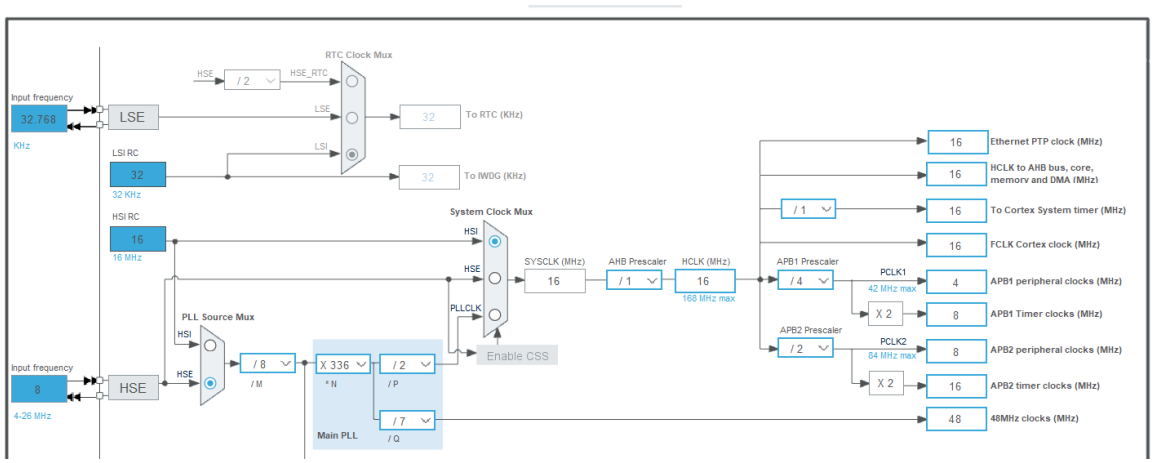


Adım 4: Saat Ayarları (Clock Configuration)

Bu projede dahili osilatörü kullanacağız.

- Üst sekmelerden **Clock Configuration**'a gelin.
- *Bu deney için varsayılan saat ayarları (16 MHz civarı) yeterlidir, PLL ayarlarına dokunmanıza gerek yoktur.*





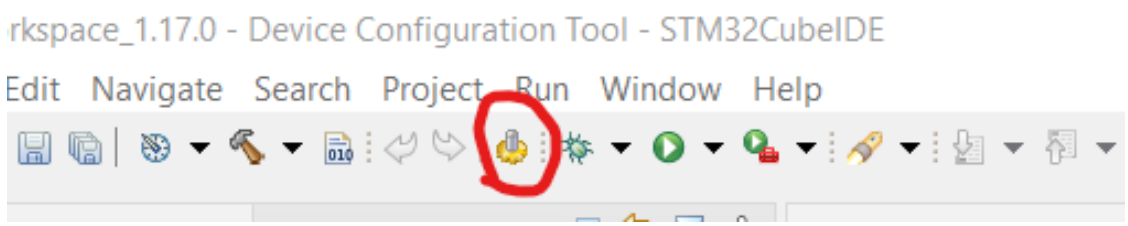
BİLGİ KUTUSU: HSI (Dahili Saat) Nedir?

Mikrodenetleyicinin kalbi olan saat sinyalini üretmek için iki yol vardır:

1. **HSE (High Speed External):** Kartın üzerindeki metal kristaldir. Çok hassas ve hızlıdır.
2. **HSI (High Speed Internal):** Çipin içindeki kendi osilatörüdür. Kristal kadar hassas değildir ama ek donanım gerektirmez. Basit uygulamalar için **HSI** idealdir.

Adım 5: Kod Üretimi (Code Generation)

- Ayarlar bittikten sonra üst menüdeki **"Device Configuration Tool Code Generation"** (Sarı dişli simgesi ⚙) butonuna basın.
- "Code is successfully generated" uyarısını aldığınızda main.c dosyanız açılacaktır.



4. Yazılım ve Algoritma

4.1. Çalışma Mantığı (Algoritma):

Geliştireceğimiz yazılım, sonsuz bir döngü içerisinde şu adımları sırasıyla gerçekleştirecektir:

1. **İleri Yön:** LED'ler 0'dan 7'ye kadar sırayla yanıp sönecektir.
2. **Geri Yön:** LED'ler 7'den 0'a kadar sırayla yanıp sönecektir.
3. **Buton Kontrolleri:** Her LED yanmadan hemen önce Switch'lerin (Anahtarların) durumu kontrol edilecektir:
    - **SW0 (PE13) Basılıysa:** Animasyon durur, tüm LED'ler yanar.
    - **SW1 (PE14) Basılıysa:** Animasyon olduğu yerde donar (Pause).
    - **SW2 (PE15) Basılıysa:** Animasyon hızı artar.

YAZILIM KÖŞESİ: Kullandığımız HAL Fonksiyonları Bu deneyde donanımı kontrol etmek için şu 3 temel fonksiyonu kullanacağız:

- **HAL_GPIO_WritePin(Port, Pin, Durum)**: Pini Lojik-1 (SET) veya Lojik-0 (RESET) yapar. LED yakmak/söndürmek için kullanılır.
- **HAL_GPIO_ReadPin(Port, Pin)**: Pinin durumunu okur. Anahtarın açık mı kapalı mı olduğunu (1 veya 0) buradan öğreniriz.
- **HAL_Delay(ms)**: İşlemciyi belirtilen milisaniye kadar bekletir. LED'in yanık kalma süresini (hızını) bununla ayarlarız.

4.2. Kaynak Kod (main.c):

- Kod yazma işlemine main.c dosyası içerisinde devam ediyoruz.

1. Tanımlamalar (Global Variables): LED'leri döngü (for loop) ile kontrol edebilmek için pin numaralarını bir diziye (array) atıyoruz. *(Bu kodu /* USER CODE BEGIN 0 */ etiketinin arasına yazınız).*

```
/* USER CODE BEGIN 0 */
// LED Pinlerini diziye atarak döngü içinde kullanabiliriz.
uint16_t LED_PINS[] = {
    GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3,
    GPIO_PIN_4, GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7
};

uint32_t beklemeSuresi = 250; // Başlangıç hızı (ms)
/* USER CODE END 0 */
```

2. Ana Döngü (Main Loop): Programın sürekli çalışacağı sonsuz döngü bloğu. *(Bu kodu /* USER CODE BEGIN WHILE */ etiketinin arasına yazınız)*

```
/* USER CODE BEGIN WHILE */
  while (1)
  {
      // --- 1. DÖNGÜ: İleri Yön (LED0 -> LED7) ---
        for(int i = 0; i <= 7; i++)
      {
          // KONTROL 1: SW0 (PE13) - Tümünü Yak
               // Switch açık olduğu sürece (SET), tüm LED'leri yak ve bekle.
               while(HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_13) == GPIO_PIN_SET)
          {
                     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_All, GPIO_PIN_SET);
               }

          // Sıradaki LED'i Yak
               HAL_GPIO_WritePin(GPIOE, LED_PINS[i], GPIO_PIN_SET);

          // KONTROL 2: SW1 (PE14) - Durdur / Pause
               // Switch açık olduğu sürece kod burada takılı kalır.
               while(HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_14) == GPIO_PIN_SET);

          // KONTROL 3: SW2 (PE15) - Hız / Speed
               if(HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_15) == GPIO_PIN_SET)
                     beklemeSuresi = 50;  // Hızlı (50ms)
               else
                     beklemeSuresi = 250; // Normal (250ms)

          // Bekleme ve Söndürme
               HAL_Delay(beklemeSuresi);
               HAL_GPIO_WritePin(GPIOE, LED_PINS[i], GPIO_PIN_RESET);
        }

      // --- 2. DÖNGÜ: Geri Yön (LED7 -> LED0) ---
        for(int i = 7; i >= 0; i--)
      {
          // SW0 Kontrolü (Tümünü Yak)
               while(HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_13) == GPIO_PIN_SET){
                     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_All, GPIO_PIN_SET);
               }

          // LED Yak
               HAL_GPIO_WritePin(GPIOE, LED_PINS[i], GPIO_PIN_SET);

          // SW1 Kontrolü (Durdur)
               while(HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_14) == GPIO_PIN_SET);

          // SW2 Kontrolü (Hız)
               if(HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_15) == GPIO_PIN_SET)
beklemeSuresi = 50;
```

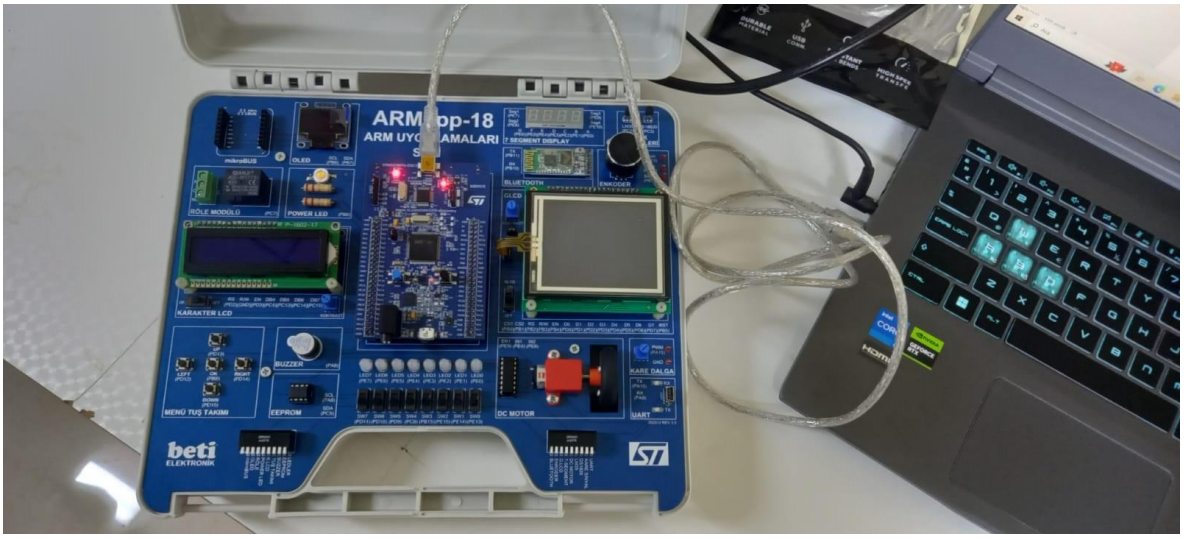
```
            else beklemeSuresi = 250;

        // Bekle ve Söndür
            HAL_Delay(beklemeSuresi);
            HAL_GPIO_WritePin(GPIOE, LED_PINS[i], GPIO_PIN_RESET);
        }

    /* USER CODE END WHILE */
```

5. Pratik Uygulama ve Test Adımları

Yazdığımız kodun STM32 kartına yüklenmesi ve deney seti üzerinde test edilmesi için aşağıdaki adımları izleyin.

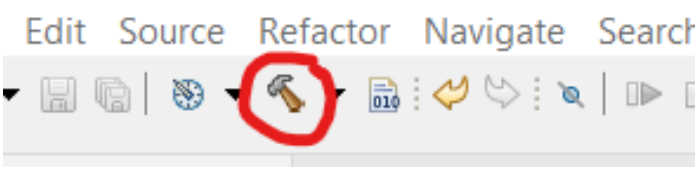
Adım 1: Fiziksel Bağlantı

1. ArmApp18 deney seti üzerindeki **STM32F4 Discovery** kartının üst kısmında bulunan **mini-USB (USB ST-LINK)** girişine kabloyu takın.
2. Kablonun diğer ucunu bilgisayarınızın USB portuna bağlayın.
3. Kart üzerindeki **PWR** (Güç) LED'inin yandığını ve bilgisayarınızın donanımı tanıdığını doğrulayın.

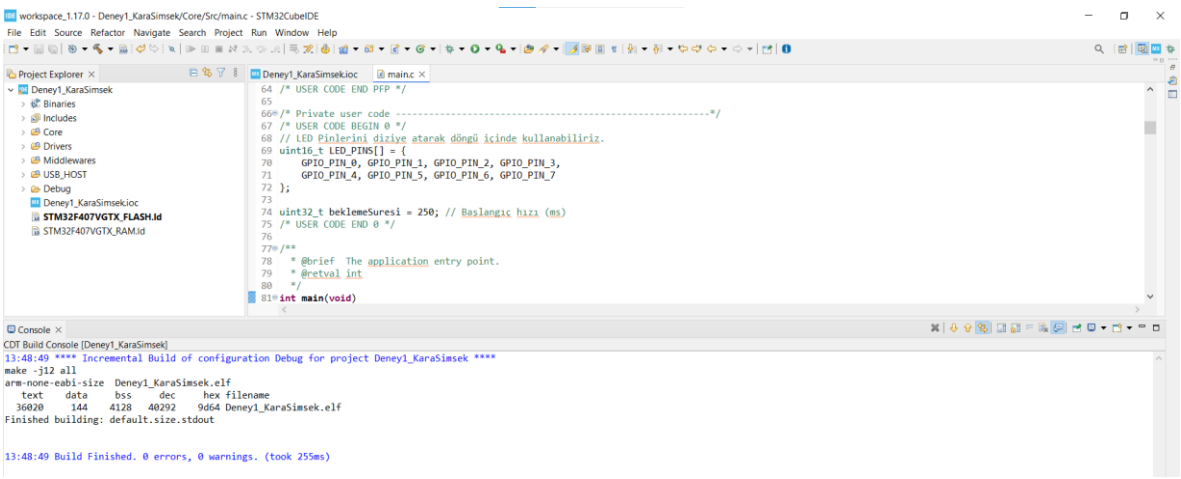


Adım 2: Derleme ve Yükleme (Build & Run)

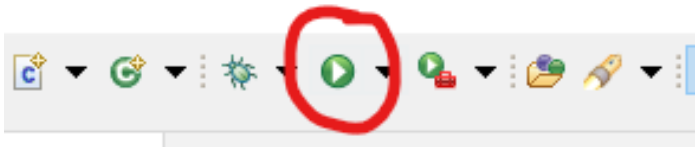
1. **Derleme:** STM32CubeIDE üst menüsündeki **"Çekiç" (Build)** ikonuna tıklayın (veya Ctrl+B).



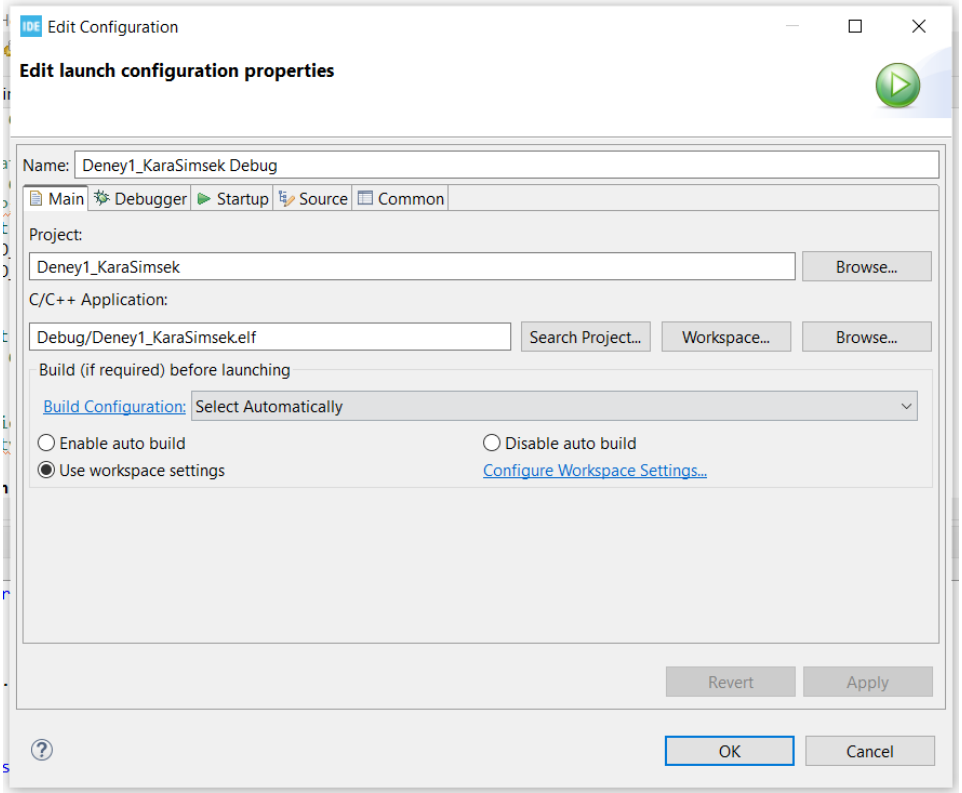
- *Console penceresinde "0 errors, 0 warnings" mesajını görmelisiniz.*



2. **Yükleme:** Derleme hatasız ise, yeşil **"Oynat" (Run)** ikonuna tıklayın.

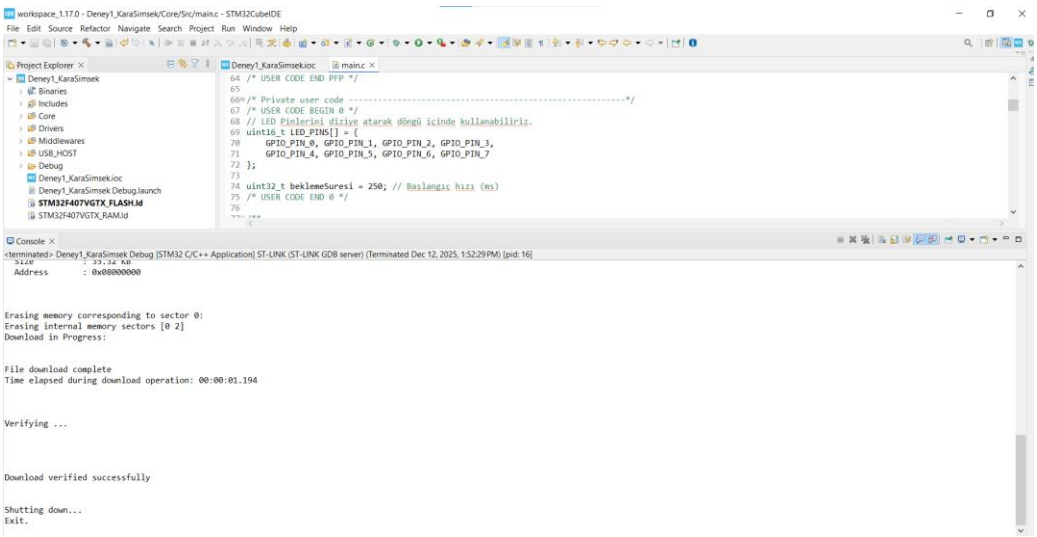


- *İlk yüklemede "Edit Configuration" penceresi açılırsa "OK" diyerek geçin.*



- *ST-LINK Firmware uyarısı gelirse "Upgrade" işlemini yapın.*

3. **Onay:** Console penceresinde **"Download verified successfully"** mesajını gördüğünüzde kod karta yüklenmiş demektir.

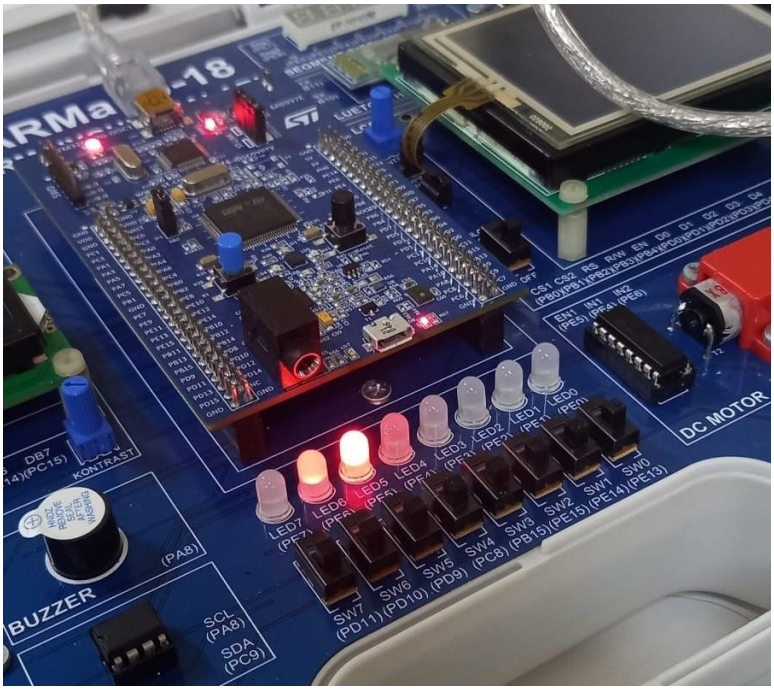


Adım 3: Deneyin Gözlemlenmesi (Test) Kod yüklendikten sonra ArmApp18 seti üzerindeki **LED** ve **Switch** modüllerini gözlemleyin:

- **Varsayılan Durum:** Switch'ler kapalıyken 8 LED sırasıyla sağdan sola ve soldan sağa (Kara Şimşek efektiyle) yanıp sönmelidir.
- **Hız Testi (SW2 - PE15):** En sağdaki anahtarı açtığınızda animasyonun hızlandığını gözlemleyin.
- **Durdurma Testi (SW1 - PE14):** Ortadaki anahtarı açtığınızda o an yanan LED'in sabit kaldığını (animasyonun donduğunu) gözlemleyin.
- **Tümünü Yakma (SW0 - PE13):** En soldaki anahtarı açtığınızda tüm LED'lerin aynı anda yandığını doğrulayın.

6. Sonuç ve Değerlendirme

Bu laboratuvar çalışmasında, **STM32F407** mikrodenetleyicisi ve **STM32CubeIDE** kullanılarak temel Giriş/Çıkış (GPIO) işlemleri başarıyla gerçekleştirilmiştir.

Elde Edilen Kazanımlar:

1. **GPIO Çıkış Kontrolü:** HAL_GPIO_WritePin fonksiyonu ile 8 adet LED belirli bir algoritma dahilinde kontrol edilmiştir.
2. **GPIO Giriş Okuma:** HAL_GPIO_ReadPin fonksiyonu ile dış dünyadan gelen anahtar (switch) sinyalleri okunmuş ve programa yön verilmiştir.
3. **Algoritma Mantığı:** Donanım tabanlı hız kontrolü, durdurma ve durum değiştirme senaryoları gömülü yazılım ile hayata geçirilmiştir.